

AMD Am9080 on FPGA and DOOM Application

Group 10

Authors:

Fabian Nunez	**Jereme Crump**	**Daniel Evans**	**Abram Tadros**
Computer Engineering	*Computer Engineering*	*Computer Engineering*	*Computer Engineering*

Review Committee:

Dr. Mike Borowczak	**Dr. Saleem Sahawneh**	**Dr. George Atia**	**Dr. Hadi Kamali**
ECE Associate Professor	*ECE Lecturer*	*ECE Professor*	*ECE Assistant Professor*

Mentor:

Dr. Chung Yong Chan

Table of Contents

Chapter 2 | Project Description..... 3

2.1.1 Project Background & Motivation.....3

2.1.2 Prior Work.....4

2.2.1 Goals..... 5

2.2.2 Objectives..... 6

2.3 Description of Features & Functionalities..... 7

2.5 Engineering Specification Tables..... 8

2.6 Prototype Illustration..... 10

2.7 Hardware Design..... 12

2.8 Software Design.....13

2.9 House of Quality.....14

Chapter 10 | Project Administration.....15

10.1 Budgeting & Financing.....15

10.2 Initial Project Milestones..... 16

Appendices

Table 1. Overall System Specifications - Page 7
Table 2. Component Specifications - Page 12
Table 3. Bill of Materials - Page 16
Table 4.1 Senior Design 1 Project Milestones - Page 17
Table 4.2 Senior Design 1 Project Design Milestones - Page 17
Table 4.3 Senior Design 2 Project Design Milestones - Page 18
Figure 1. Prototype Illustration - Page 11
Figure 2. Hardware Design Diagram - Page 13
Figure 3. Software Design Diagram - Page 14
Figure 4. House of Quality - Page 15

Chapter 2 | Project Description

2.1.1 Project Background & Motivation

The AMD Am9080 is the first microprocessor created by Advanced Micro Devices (AMD) as a clone of the Intel 8080 utilizing the Intel 8080 instruction set architecture. Originally, the Am9080 served its role as a second-source device that helped innovate computing. Despite the Am9080's historical significance, it lacks in running modern or complex software programs on the 8-bit CPU architecture. This microprocessor functions using a multi-cycle state machine with a 16-bit address bus which limits the memory to a 64 KB RAM. Even though these specifications were considered great and functional during the early technology days, running graphically intense games, even a modified DOOM will cause hurdles throughout the way. We have two main challenges that the team needs to solve. First, we need to accurately design an Am9080 and run it on an FPGA with bank switching in order to have 4 MB addressable memory rather than 64 KB and to accurately verify the RTL design. Second, we need to build an optimized, bare metal version of DOOM relying only on fixed-point arithmetic.

At UCF the senior design projects from the ECE department allow us undergraduate students to use all the skills and tools learned through our undergraduate tenure to achieve a common goal with other students. As a group of computer engineering students we must combine our strengths in RTL design, embedded systems, and digital logic to complete this project that touches both hardware and software. We saw the challenge and applied to the project knowing that it would push us past the curriculum we've seen in our coursework and with AMD sponsoring the project it gives us all a great opportunity to work directly with an industry giant on a project with real technical depth. Running a DOOM-style application on an Am9080 processor has not been done before by a team and the closest feat being on later architectures like the Z80 and 6502 using heavily optimized software.

2.1.2 Prior Work

Designing a microprocessor and running it on an FPGA and running a program on it has been done numerous times in the past. In fact, it is considered to be a standard computer engineering capstone project. However, our project is unique in a couple of different ways. First, most of the current projects include the design of a 32-bit processor, usually utilizing open source instruction sets like RISC-V. This allows most C programs to seamlessly run on the CPU without much, or any, software optimizations. However, we are recreating an 8-bit microprocessor that was initially created for simple logic and data manipulation. So, in order to run a math intensive graphical application, we will need to utilize software optimization techniques. Second, we have

not found another group that recreated this processor and ran a DOOM-style game on it. Our goal is to be the first to do so.

A relevant project is the HOLY CORE, an open source project hosted on Github (0BAB1/HOLY_CORE_COURSE) that consists of a RV32I(M) RISC-V core with a bare metal DOOM port (HOLY_CORE_DOOM) built on a RISC-V Doom codebase. This project shows how a fully custom made soft core can be taken from RTL and software to run a real graphical game. RV32I(M) is a modern 32-bit ISA with a wider address bus that can reach more memory than the Am9080 can without bank switching. HOLY CORE also did not have to solve the two biggest problems our project must solve: rendering math on an 8-bit CPU without floating point units, multiplication, or division, and extending a 16-bit address bus into a multi megabyte space. It also has no FPS or “turbo mode” benchmark and its DOOM demo needs an exact hardware match, making it hard to reproduce. This motivates us to thoroughly document our bank switching scheme to make our system reproducible.

Another iteration of DOOM called POOM was created as an 8-bit version. This version was recreated specifically to run on the Pico-8. DOOM originally was developed to run on a 32-bit microprocessor with at least 4096 KB of RAM (4 MB). The challenge in our project is to modify (or recreate if needed) DOOM to be able to run on an 8-bit microprocessor. Originally, DOOM was an upgraded version of the game called Wolfenstein. This game essentially was a 3D game that utilized 2D raycasting techniques that did not require a 3D game engine. In order to recreate DOOM for an 8-bit microprocessor, we are going to need to utilize similar techniques to wolfenstein raycasting but also eliminate system calls with assembly instructions since there will not be an operating system.

Another project that is also relevant to the memory expansion portion of the “C64 on an FPGA”, documented on a blog by Johan Steemkamp. This blog details how Steemkamp was able to recreate a C64 system on an FPGA. The C64 system is very similar to the system we want to create: an 8-bit cpu with 64 KB of memory addressing, with the ability to expand the memory via bank switching. In Steemkamp’s project, he uses bank switching logic in verilog to expand the memory. This project’s knowledge and techniques will provide useful advice on how we can accomplish bank switching on the FPGA we are using. Obviously, the C64 implementation will differ greatly from what we are doing, but it is still a good basic example of the process we could follow.

Overall, throughout ECE projects across multiple schools & hardware engineers, designing a CPU using an FPGA and running a game on it is the go-to project to enter into the hardware realm. There have been multiple projects implemented utilizing different open source instruction set architectures and games. And for us, we would like to be the first to do so with AMD’s first CPU and the infamous game DOOM.

2.2 Goals and Objectives

2.2.1 Goals

The goal of recreating the Am9080 is to be the first team to run a modified DOOM-style game on it. To achieve this, our project is broken down into a mix of hardware and software co-design efforts. In order to complete this project, we have both hardware and software requirements that must be met along with stretch goals depending on our project progress.

- **Basic Goals:**
  - Recreate a functioning Am9080-compatible microprocessor on an FPGA.
  - House the recreated CPU on a hardware platform capable of supporting memory, storage, a display, and user input.
  - Expand the system's addressable memory beyond the Am9080's native 64 KB limitation to 1MB.
  - Run a program coded in C using the recreated processor.

- **Advanced Goals:**
  - Verify that the recreated Am9080 behaves correctly against the original 8080 instruction set.
  - Provide a configurable Turbo Mode that runs the system faster than the original Am9080's clock speed.
  - Use bank switching logic to expand the Am9080’s RAM addressing to 4 MB.
  - Demonstrate a playable, DOOM-inspired graphical application running natively on the recreated Am9080.
  - Run the completed DOOM-inspired application on the Am9080 FPGA implementation in standard mode and Turbo mode.
  - Create a 1-3 level map with 1 first person shooter with the goal of eliminating 3-5 sprites per level before finishing the map. The first person shooter will have 1-2 weapons to choose from, each with unlimited ammo.

- **Stretch Goals:**
  - Provide a pin-compatible interface so the FPGA implementation can physically replace an original Am9080 chip via socket or interposer.
  - Design a custom PCB to utilize bank switching to expand the hardware’s memory to 4 MB.
  - Achieve gameplay performance exceeding 15 frames per second.

2.2.2 Objectives

- **Basic Objectives:**
  - Design and implement the Am9080 instruction set architecture in HDL.

    - Confirm basic functional correctness of the Am9080 design by cross-compiling and running a simple C program on the FPGA implementation.
    - Interface the FPGA development board, which includes 4MB of RAM and flash storage, with a microSD card for data storage, a USB port for keyboard input, and a VGA output for driving a 320 x 200 to 640 × 480 display.
    - Implement a bank-switching module that divides the 4 MB RAM into 16 KB banks, with the active bank selected by the CPU through a bank-select register.
    - Cross-compile C source code to 8080 machine code and execute it directly on the Am9080 FPGA implementation without an operating system.
    - Create a graphical application to show multiple enemy sprites and an FPS player.
    - Create a graphical 3D map utilizing 2D raycasting techniques.

- **Advanced Objectives:**
    - Perform functional verification of the RTL design against the Am9080 instruction set to confirm cycle-accurate behavior.
    - Implement a configurable Turbo Mode that allows the FPGA clock to run at a user-selectable frequency higher than the standard 2–4 MHz range.
    - Develop a simplified, DOOM-inspired graphical application using grayscale rendering, a 160x120 internal render resolution, and fixed-point arithmetic with precomputed trigonometric lookup tables, since the Am9080 lacks MUL/DIV opcodes and a floating-point unit.

- **Stretch Objectives:**
    - Add a CPU socket to the motherboard to enable a true pin-compatible, drop-in replacement for an original Am9080 chip.
    - Optimize the rendering pipeline and gameplay loop to sustain a frame rate greater than 15 fps.
    - Interface a custom RAM PCB with bank switching lines to the CPU socket to allow memory expansion.

2.3 Description of Features & Functionalities

Overall Functionality: The overall objective of this project is to be able to run a custom built DOOM-style game on AMD's original CPU, the Am9080. Since this microprocessor was designed in the early 1970s, as an 8-bit CPU, it is not meant to handle 3D graphical applications like modern games. To complete this project, we will utilize a system that hosts the FPGA and develop a bank switching memory system in order to achieve a 4 MB pageable memory, and our custom software game built strictly for 8-bit architectures.

Software Functionality: The software game engine will be written in the C programming language and cross-compiled to 8080 machine code to run directly on the CPU without an

operating system. Since the Am9080 doesn't have MUL or DIV opcodes and it also does not have a floating-point unit, we will need to solely rely on fixed point arithmetic for calculations regarding positions and distances. However, as trigonometric operations are still needed, these can be solved by having pre-existing lookup tables/arrays with pre-calculated values. This essentially replaces heavy trigonometric math by simply using a function call.

Memory Functionality: The Am9080 contains a 16-bit address bus, which essentially limits the memory to 64 KB. Since a graphical game requires significantly more memory, we are going to need to create a memory expansion system to allow us to have at least 4 MB of pageable memory. This will be done using a technique called bank switching. Bank switching allows us to go past the 64 KB memory space by dividing the RAM into banks. Essentially, each bank stores a block of memory. The software instructs the bank controller via I/O writes to switch which bank is currently mapped in the CPU's side depending on what the program needs. This technique allows us to have 4 MB of memory stored, however, since the CPU has a 16-bit address bus, it only has 16 address lines so it can only access 64 KB (2^16 = 64K) of memory at any given moment.

FPGA Functionality: The FPGA functions as the processing brain of our whole project by including our synthesized design of the Am9080 microprocessor core. The Am9080 features an 8-bit data bus and a 16-bit address bus which allows it to address 64 KB of memory. By utilizing a high speed FPGA, we will be able to execute instructions at standard clock speeds but also include a configurable Turbo Mode feature which will allow us to run our program at a higher clock speed which allows for better overall performance. The Turbo Mode will essentially allow the system to process the raycasting math faster during gameplay, potentially allowing for higher frames per second.

2.5 Engineering Specification Tables

Table 1. Overall System Specifications

Parameter	***Target value***	***Justification***
Frame Rate	≥ 15 Frames per Second	Aligned with AMD's target in the project brief, which specifies 15 FPS in turbo mode, and represents the threshold at which the demo

		can be considered a “playable demonstration” rather than merely a bare-minimum implementation.
Frame Budget Utilization	60-85% in standard mode	At 15 fps and 2 MHz the frame budget is 133,333 T-states. A target of 60 to 85% corresponds to 80,000 to 113,333 cycles of loop time per frame.
Pageable Memory Capacity	≥ 4MB	Required to satisfy the memory expansion goal. The additional memory provides the necessary storage for map geometry, texture assets, and frame buffer data.
Standard Mode Clock Speed	2 MHz - 4 MHz	We chose to stay with the original Am9080 clock range which keeps accurate instruction timing
Turbo Mode Clock Speed	≥ 2x Standard clock speed	The brief requires a selectable turbo mode. 2x is set as the minimum multiplier needed to produce a meaningful jump in frame rate over the standard mode.
Internal Render Resolution	{(80 x 60) to (320 x 200)}, targeting 160 x 120	Raycasting cost scales with resolution. At 15fps and 2MHz the frame budget allows for ~830 T-states per column which supports 160 columns; 320 columns would halve that budget and is reachable only in turbo mode. Below 80 columns is where the wall geometry becomes too flat to read as a 3D scene. 160 x 120 sits where the pixel work fits the standard mode budget while also matching the 4:3 VGA output aspect ratio

Physical Display Resolution	{(320 x 240) to (800 x 600)}, targeting 640 x 480	The physical resolution must be an integer multiple of the internal render resolution. 640x480 is a clean 4x integer upscale of our targeted 160x120 internal resolution and is also the most widely supported VGA timing mode.
Physical Display Refresh Rate	30-60 Hz	Standard VGA refresh rate to ensure broad monitor compatibility
User Input Delay	≤50ms	Input lag above 100ms is too noticable. 50ms leaves a good margin while accounting for the raycasting render loop on the Am9080
Memory Paging Window	~[16 - 32] KB Memory Bank	The paged window must leave enough of the Am9080's 64KB address space resident to hold the game loop, stack, LUT's, memory-mapped I/O, and framebuffer which is an estimated 22-30KB. The lower bound of 16KB is set by the bank select register: 4MB / 16KB = 256 banks which is the largest count an 8-bit I/O write can address. Our implementation targets the lower end of this range to maximize resident space
Memory Stall Fraction	≤ 20% of total cycles	The portion of CPU cycles spent in WAIT with READY deasserted. Memory stalls are measured by counting cycles where READY is low. 20% bounds the overhead at roughly 1.25x the architectural CPI, keeping the effective throughput within the stated range
Cost	≤ $1500	Based on the budget provided

		to us by the AMD team.
Instruction Throughput	180k-250k instr/s at 2 MHz	The Am9080 render loop has an architectural average of ~6-8 T-states per instruction, dominated by register moves(5), memory operands (7), and branches (10). The target range corresponds to an effective 8-11 T-states once DDR2 latency and bank-switch WAIT states are included.
CPU Cycles Per Instruction	4 - 18	Per the Am9080 Datasheet instruction set summary, the instructions with the lowest number of clock cycles take ~4 cycles and the instructions with the highest number of clock cycles being ~18 cycles
Zombieman Player	1	Zombieman is the first person shooter player that eliminates enemy sprites to finish the level.
On Screen Sprites	3-5	Sprites are enemy characters on the screen. Zombieman
Zombieman Health	0-100	The FPS player and sprites will both have a health level from 0 to 100.
Weapons	1-2	FPS Player will have 1-2 weapons with unlimited ammo.
Game Storage Size	256 KB to 3 MB	Estimated from the asset list: grayscale wall textures, enemy & weapon sprites, 1-3 level maps, compiled game code. The lower bound reflects a minimum playable build while the upper bound leaves headroom for additional levels and textures

2.6 Prototype Illustration



Figure 1. Prototype Illustration

Table 2. Component Specifications

Component	***Parameter***	***Specification***
FPGA Artix-7	Logic Utilization	The FPGA will house the Am9080 core, memory, I/O, Display controller.
BRAM	Storage Capacity	4860 Kbits on chip RAM available for code, stack, buffers.
DDR2 RAM	Usable Capacity	128 MiB (134 MB) available on chip for DDR2 RAM.
QSPI Flash	Storage Capacity	16 MB of flash memory available to keep game files stored.
Memory Controller	Bank Window	~16KB window mapped for bank switching to achieve 4 MB pageable memory.
Display Output	Resolution	640 x 480 @ 60 Hz via 12-bit VGA output.
Logic Slices	FPGA Size	15,850 FPGA Logic Slices that allow for the RTL design to be on the chip.
PMOD Connectors	External Connections	4 Pmod connectors that can be used for external RAM.

2.7 Hardware Design Diagram

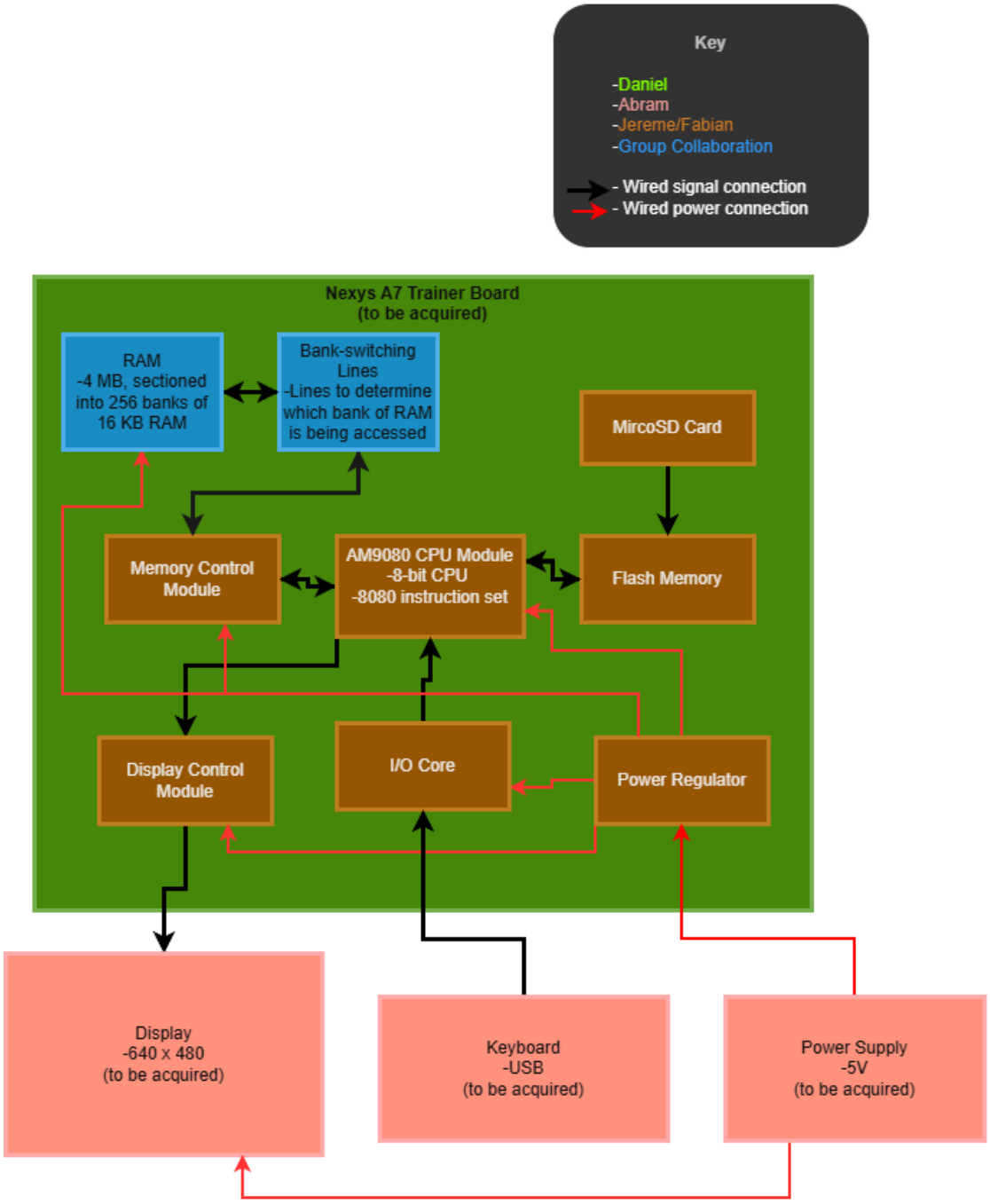


Figure 2. Hardware Design

Figure 2 illustrates the hardware architecture of our recreated Am9080 which sits on the Nexys-A7. The arrows show the direction of connection which are separated by color to show the type of connection; box colors indicate which team member is responsible for that subsystem.

2.8 Software Design Diagram

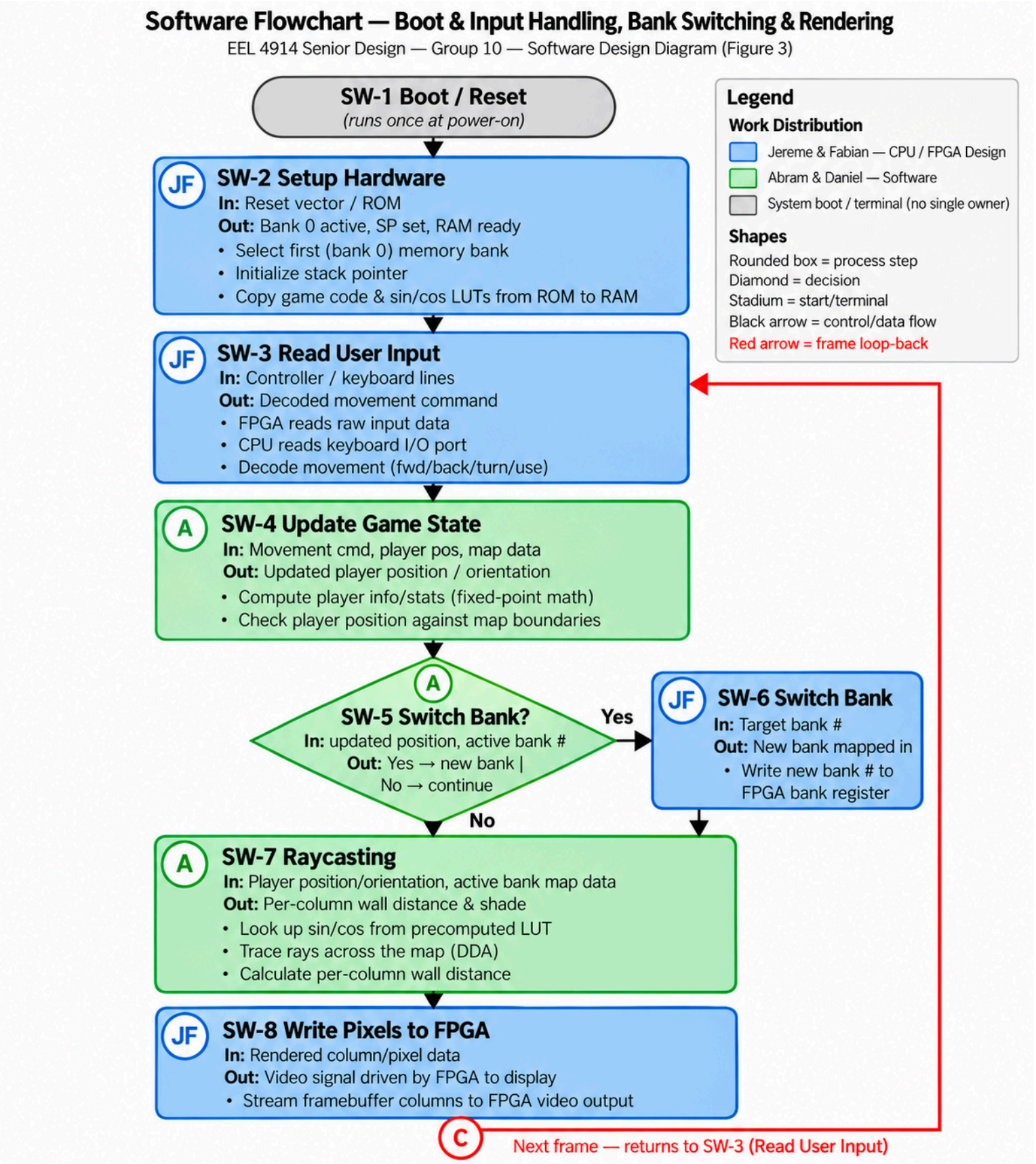


Figure 3. Software Design

2.9 House of Quality

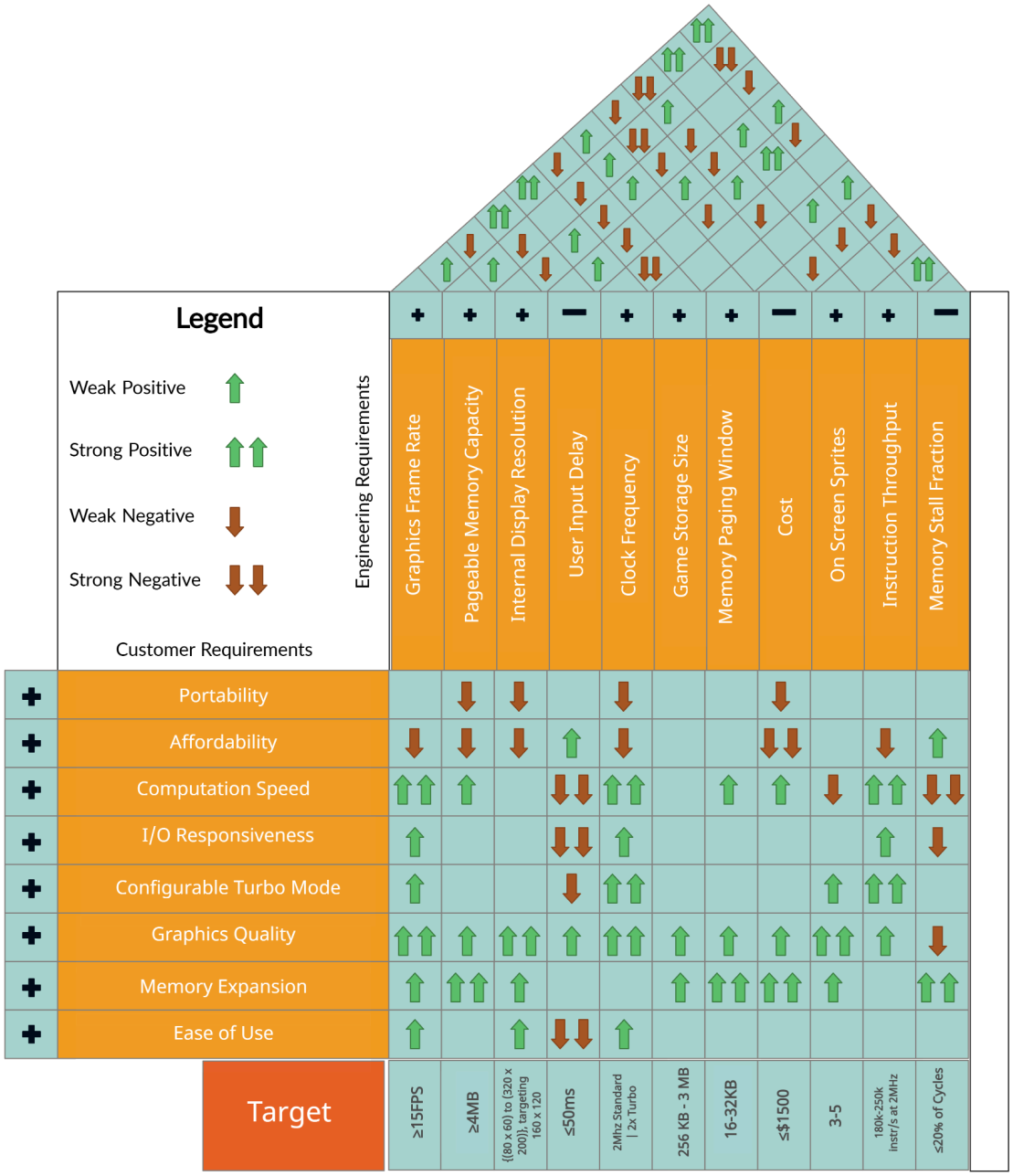


Figure 4. House of Quality

Chapter 3 | Research and Investigation

3.1.1 Am9080 Implementation

Before choosing the specific hardware for this project, our team had to decide what kind of platform would house the Am9080. Today there are four feasible ways to run Am9080 software: use original silicon, emulate the processor in software on a microcontroller, implement the processor in an FPGA on a custom PCB, or implement it in an FPGA on a commercial development board. Each approach meets the project's goals to a different degree but the last one is the most concrete implementation. The main factors that differentiate these approaches were whether the approach recreates the CPU in hardware, whether it can support 4MB of bank switched memory, can support turbo mode, how much time it takes to develop, and how much risk it has.

3.1.1.1 Original Silicon

Building a system around an original Am9080 would be the most accurate option. The Am9080 was made as a pin-for-pin replacement/clone of the Intel 8080. It is fabricated in n-channel silicon-gate MOS technology. The Am9080 needs three voltage supplies: +12V, +5V, and -5V each within ±5% for the commercial grade [Am9080A datasheet] which unlike modern logic that runs from a single low voltage supply.

The processor also cannot run on its own. 8080 architecture uses a 2 phase, non-overlapping clock that's driven at MOS voltage levels. In the original older systems this clock came from a dedicated clock generator chip called the 8224. It also used a system controller called the 8228, which latched the processor’s status byte and produced the memory and I/O read and write enable bits. Because of this, a minimum working system needs the CPU, both supporting chips, a 3 rail power supply, and external memory.

The next limitation is the clock speed and memory. The datasheet lists a minimum clock period of 480ns for the Am9080A and 250ns for the fastest grade (Am9080A-4) which translates to about 2MHZ and 4MHz respectively. Since the silicon cannot run faster than its rated clock speed, the project’s turbo mode objective cannot be met with the original silicon hardware. We would also need some external bank switching logic to reach 4MB of memory since the processor only has a 16-bit address bus.

Finally, the original Am9080A parts have been out of production for decades and are only available as surplus stock so if a chip is damaged we have no way of getting a replacement. The main drawback of this implementation is that it doesn’t recreate the processor at all, AMD’s project requirements ask us to recreate it and buying the original chip skips that entirely.

3.1.1.2 Software Emulation on Microcontroller

The second implementation option is to write or use an open source software emulator that interprets 8080 class instructions and runs on a modern microcontroller. A modern microcontroller like the Raspberry Pi RP2040 runs at 133MHz with 264KB of SRAM which has much better performance than the original Am9080 running at 2MHz. An emulator can give the CPU access to lots of memory and can also run at any chosen speed.

There are several weaknesses with this implementation. The main one is that the emulator is just a C program that imitates the Am9080 and produces no RTL. Without a RTL design we can't verify anything against the datasheet which rules out one of our advanced objectives: functional verification of the RTL design against the Am9080 instruction set to confirm cycle-accurate behavior.

The emulator also can't measure several of the engineering specifications in Table 1. Our standard mode target of 2-4MHz and our Turbo Mode target of at least 2x standard speed both assume a real hardware clock. The Memory Stall Fraction target of ≤ 20% of total cycles is measured by counting the cycles where READY is low while the CPU waits on memory. An emulator has no READY signal and no real RAM latency, so every memory access finishes instantly and would net a 0% stall fraction. Because of this, the instruction throughput target of 180k-250k instr/sec, which accounts for DDR2 latency and bank switching WAIT states, would look way better in the emulator than in real hardware. The Frame Budget Utilization target of 60%-85% would also be redundant. The emulator can only model a bank select register in software and can't test the decoder logic, bus timing, or memory controller that the real hardware needs. For these reasons, the emulator is a useful tool for building and testing the DOOM implementation before the hardware is ready and cannot serve as the final platform for our project.

3.1.1.3 FPGA on Custom PCB

The third implementation is to write the Am9080 in RTL on an FPGA mounted on a PCB designed by us. This meets the hardware goal since the processor is written in Verilog, synthesized into the FPGA's logic, and runs as real hardware. A custom PCB also gives us full control over the system layout which would allow us to include the exact memory, display and input connections the project needs.

The cost of this option is a much harder hardware design. We would need an FPGA large enough like the AMD Artix-7 XC7A100T which comes in a 324-ball BGA package [AMD Artix-7 package guide]. BGA parts can't be hand soldered reliably and would usually need professional assembly which could add time and money to our schedule. The FPGA also needs several separate power rails. Texas Instruments' Artix-7 reference design asks for a 0.95V-1.0V core

supply capable of up to 6.1A, a 1.8V auxiliary supply, I/O bank supplies that are adjustable from 1.2V to 3.3V, and a separate supply for DDR memory [TI TIDUE63]. The board would also need its own JTAG programming circuit, configuration flash, and clock source.

Each of these is a full design task that takes lots of time and risk where any error would require a redo of the board before we could even start RTL testing. Because the processor design is the main focus we talked with our advisors and got the significant PCB requirement waved for our project. Since it is not required of us we decided to treat any type of custom PCB as a stretch goal after the processor is proven to work on existing hardware.

3.1.1.4 FPGA Development Board

The last option is to implement the Am9080 RTL on a commercial FPGA development board. A development board houses a working FPGA on a PCB designed specifically for it with tested power regulation, external memory, clocking, user interfaces like LED's, buttons and switches, and circuitry for configuration. Most boards also include peripherals that line up with this project's needs such as VGA output, a USB keyboard and mouse interface, onboard RAM and flash storage, as well as expansion header pins for extra modules if needed.

With a development board

3.1.1.5 Technology Selection Summary

- Table comparing each implementation
- Summary paragraph explaining which approach we chose and why

3.1.2 FPGA Board Selection

3.2 Communication

3.2.1 HDMI

3.2.2 DisplayPort

3.2.3 VGA

3.2.4 VGA

3.2.5 PS/2

3.2.6 USB

3.3 Software Application Research

The primary software goal of this project is to design and execute a graphical 3D raycasting game application inspired by early age games such as DOOM and Wolfenstein 3D running natively on a recreated 8-bit AMD Am9080 processor core. To prove the microarchitectural accuracy, throughput, and memory management of the FPGA system, the software game must execute arithmetic computation and memory manipulation while maintaining a playable frame rate on a standard 640 x 480 at 60 Hz VGA monitor.

When designing the software architecture for the game application, there are some engineering parameters that need to be satisfied. The following are:

- **Target Frame rate:** Minimum of 15 frames per second, in Turbo Mode, during active camera and player movement. Turbo Mode should have the CPU execute instructions at a faster clock frequency which will increase FPS.
- **Internal Render Resolution:** 160 x 120 pixels (19,200 total pixels per frame) which is then scaled by 4 in hardware to fill the 640 x 480 display monitor.
- **Pageable Memory and Bank Window:** Total pageable memory capacity of 4 MB accessed through a 16-32 KB banking window, with the rest of the Am9080 memory in FPGA BRAM to hold the game loop, game LUTs, memory mapped I/O, video framebuffer.
- **Game Content & Storage Size:** The total game size will be ~3 MB housing 1 to 3 level, grayscale wall textures, 1 first person zombieman player with 0 - 100 health and 1-2 selectable weapons with unlimited ammo, and 3-5 enemy sprites per level that must be eliminated to advanced.

As shown in the Software Design flowchart in Chapter 2, our software loop runs through 8 stages: boot/reset, setting up the hardware to select the first bank, initialize the stack pointer, and loading the game code with its LUTs to the RAM when flashed.

3.3.1 Development Environment: Programming Languages & Version Control

Add an overall summary here

3.3.1.1 Python

Translating Python to 8080 assembly code is a challenging ordeal. For one, Python doesn't get compiled into any kind of assembly language normally, it is instead interpreted by a runtime environment, which interprets the code while it's running. This would mean replicating the runtime environment in lower-level code, which is its own issue. Additionally, Python has many

features that are dynamic, meaning some variable types can change while running, limiting how that feature can be implemented in assembly.

There are existing tools that can help translate Python into some other language, like C, and then into an assembly language. However, some of the dynamic features of Python are unable to be translated and will fail, leading to even more issues. One successful implementation of converting Python to x86 assembly was done by Ben Hoyt and is documented on his blog. His implementation took a very basic piece of Python code, one that does not use any dynamic features, and translated it into x86 assembly. Though the assembly output worked, it was still outperformed by an unoptimized gcc C compiler.

Overall, Python would be an unwise language to use to create our game application. It has too many issues with converting to modern assembly languages, let alone the outdated 8080 assembly. Additionally, any Python code that can be converted would not be well optimized, leading to performance issues.

3.3.1.2 Java

Java has some of the same problems as Python when trying to convert to assembly. For example, Java also doesn't compile into assembly. It instead compiles into bytecode that the Java Virtual Machine executes with the Java Runtime Environment. Like Python, there are many dependencies and complexities that make converting Java into assembly a multistep, difficult process.

One of the biggest advantages that Java has over Python is that Java requires all variable types to be declared, and they can not change during runtime. This makes converting to assembly much easier. Several Java-to-Assembly converters exist and are readily available online, most being powered by AI. For example, a Java-to-assembly converter provided by CodingFleet uses AI models to read Java code, interpret its purpose, and write out its equivalent in assembly. The biggest issue with this process is that i8080 assembly is so hardware specific that AI may not reliably translate the java code we write into its correct equivalent in i8080.

Overall, Java has some definite advantages over Python. However, the biggest issue with Java is converting it to assembly. That process is extremely complicated to do without the use of AI, and using AI to translate the code into assembly is unreliable. For this reason, we wouldn't want to use Java to create our game.

3.3.1.3 C

The C programming language has many benefits for this project. For one, C

3.3.2 Game Programming FPGA vs Emulator

There are two main environments for the game during development: a software emulator running on a personal computer and the Am9080 processor synthesized onto the FPGA. Both must execute the same instruction set, however, they do not provide the same development hardware. The emulator is useful because the program can be loaded and tested without waiting for the CPU RTL design, memory controller, and the VGA interface to be completed. The FPGA is required because the project's final objective is a physical recreation of the microprocessor. Choosing an emulator as the only target would not demonstrate the functionality of the CPU hardware that is developed by our group.

3.3.2.1 Compilers

3.3.2.2 Bare-metal Programming

3.3.2.3 Custom Linux Kernel

3.3.3 Games Graphics Libraries & Rendering Methods

Selecting the graphics method requires distinguishing the difference between a graphics application programming interface and the rendering algorithm utilized by our game. DirectX 12 and OpenGL are interfaces commonly used to submit graphical work to graphics hardware in a desktop computer while raycasting is a method for calculating the visible walls in a scene. These three tools are not interchangeable software libraries but often utilized together. Some of these tools often require a graphics processing unit (GPU) or an operating system, which will not be housed in our FPGA system implementation.

3.3.3.1 DirectX 12

DirectX 12, is a Microsoft graphics interface intended for programs that use a compatible GPU. It provides direct control over rendering methods, GPU resources, and programmable graphics. Its design is intended to reduce CPU overhead and effective use of graphics hardware and processing. Its main use is of low level access, multithreading applications, and game raytracing. For many games, especially modern games, these resources and capabilities are a great advantage since complex geometry and effects are mainly handled by hardware parallel processing.

These resources are not available in the recreated Am9080. DirectX 12 assumes a supporting software environment, graphics driver, and a GPU, while our FPGA implementation has a custom 8-bit microprocessor and a VGA display controller. Utilizing a DirectX 12 renderer would require us to also design either a custom GPU or a graphics accelerator which its result would ultimately be no longer a straightforward DOOM application on our recreated Am9080 microprocessor. Furthermore, this significant level of hardware design is most definitely not needed for a simple raycasting game.

DirectX 12 could still be used on a host development computer if our group wanted to build a modern visualization or utilize it as a debugging tool. However, it would have no advantage over a simpler SDL2 display layer already planned for the emulator. It would also make the host application more dependent on that platform than our current implementation. For these reasons, DirectX 12 is not an optimal choice for our game's final rendering system or its primary software test environment.

3.3.3.2 OpenGL

OpenGL is a widely used graphics API used to render 2D and 3D graphics on computers that have the required supported graphics drivers. Unlike DirectX 12, which is mainly used on Windows, OpenGL can be used across various operating systems. For our project, OpenGL could be used to display the game through an emulator running on a computer. Basically, it would allow us to test the graphics and compare how the game performs before running it on the Am9080 microprocessor.

However, OpenGL is not a practical option for running our game on the FPGA implementation of the Am9080. OpenGL relies on graphics drivers and supporting hardware that are normally found in modern computer systems. Since our project uses an 8-bit processor running without an operating system, these features will not be available. The Am9080 also has limited processing power and memory, which would make implementing an OpenGL graphical system almost impossible, even if we had a custom kernel supporting it. Even though OpenGL supports various operating systems, this does not mean it can run directly on our FPGA implementation.
For these reasons, OpenGL would only be useful during the development, testing, and debugging stages of the game. And since our initial plan is to utilize the SDL2 library to handle graphics and user input for our emulator implementation, adding OpenGL would introduce additional work without providing any benefit. Instead, it would be more practical to utilize raycasting techniques, that was utilized in games like Wolfenstein, to calculate the 3D environment, while the FPGA display controller will handle the video output. Therefore, OpenGL will not be used in the final implementation of the game on our recreated Am9080.

3.3.3.3 Raycasting

Raycasting was our main point of research topic to dive deep into since it is a rendering method that can be reduced to the operations that are supported by an 8-bit processor. Instead of maintaining a general three-dimensional scene and sending triangles to a GPU, a raycaster starts with a two-dimensional map and determines which wall is visible in each screen column. The result is a 3D looking image that can be calculated with 2D positions and distances. Essentially, it creates a pseudo 3D perspective from 2D mathematical calculations. This method fits the team's plan of using fixed-height walls, limited enemy sprites, limited map levels, and low internal resolution.

A simple raycaster represents the environment as a grid of cells. A value of zero defines an empty cell, while a positive value other than zero identifies wall types or texture values. The player has a position and a view direction. For each screen column, the program determines a ray direction corresponding to that column's location across the field of view. It then advances through the grid until it hits or encounters a wall cell (a positive nonzero value in the wall map array). A DDA, Digital Differential Analyzer, algorithm determines whether the next crossed grid boundary is horizontal or vertical and updates the map coordinate. This avoids sending a ray in many small distance increments and testing every intermediate position.

Once a wall has been found, the program calculates its distance from the player and chooses the height of the wall in the image. The measured distance must correspond to the distance being projected along the viewing direction rather than simply the direct length of the ray, otherwise, the projected image can show a curved wall distortion. The top and bottom of the wall slice can then be clipped to the screen. Drawing a solid slice is essentially less computationally costly than a texture slice since it does not require a texture lookup for each pixel.

The raycasting examples commonly used on modern computers perform these calculations with floating-point variables, such as the famous Lode's Computer Graphics Tutorial. Those examples are useful for understanding the algorithm, however, they are not suitable for direct execution on the Am9080. This processor has neither a floating-point unit nor multiplication and division instructions. The game must instead represent fractional values as scaled integers and replace repeated expensive mathematical computations whenever possible.
Fixed-point arithmetic is a method of representing real numbers, including decimals, where a certain number of bits are reserved for its fractional portion. For example, if 8 bits are used for the fractional portion, the number 1.0 would be stored as 256. This allows calculations such as addition and subtraction to be performed using integer operations instead of floating point arithmetic. However, multiplication is more complicated since the result must be adjusted to maintain the correct decimal value. For our project, fixed-point arithmetic is important because the Am9080 does not have a floating-point unit or even dedicated multiplication and division instructions. Although fixed-point arithmetic allows us to avoid floating-point calculations,

multiplication and division can still be done with a large number of CPU cycles. In order to reduce the processing time, we will need to limit unnecessary calculations, use smaller numerical values when possible, and rely on lookup tables for trigonometric computations. Essentially, these simple optimization techniques will be crucial for improving the performance of our raycaster on the 8-bit microprocessor.

Lookup tables can replace some mathematical operations with memory reads. In our program, we will have precomputed sine and cosine values for a fixed set of viewing directions and store the results as scaled integers. Tables for reciprocal values or wall-height conversions are also another additional optimization we can try out if the performance is not up to standard. However, lookup tables consume memory, and a table stored in another bank may introduce transfer overhead than the operation it replaces. Furthermore, tables used every frame should be small enough to remain in fixed memory or be copied into working storage.

Table 3.3.3.3.1 Comparison of raycasting operations on different systems

Calculation	Modern Desktop Implementation	Proposed Am9080 Implementation	Reasoning
Player Position	Floating point	16-bit fixed point when feasible	Avoids software FPU computations
Camera Direction	Floating point vectors	Precomputed lookup tables	Reduce trigonometric calculation
Wall-grid Traversal	DDA with floating point	Integer/fixed-point DDA	Use additions and comparisons
Wall Projection	Division by distance	Precomputed distance calculations lookup tables	Limit division computation in each column
Solid Wall Drawing	Graphics library call	Video interface writes	No desktop graphics library
Textures	Software or GPU Sampling	Texture index lookup	Limit memory and bank switching

There are several ways to improve the time required to draw each frame. The first is to reduce the internal resolution while preserving the 640 by 480 VGA output through integer scaling. The second is to use a limited number of wall textures, grayscale colors, or simple shading based on the side of the wall that was hit. The third is to limit the maximum ray distance and define a hard

coded result for rays that leave the map or that are out of the view distance. The fourth is to update common game data outside the inner rendering loop instead of recalculating it for every screen column. These changes may provide significant performance improvements without requiring the need for a full graphics engine.

A renderer that uses 160 rays per frame has an average allowance of approximately 844 CPU clock cycles per column at 2MHz and 15 frames per second. This is obtained by dividing 133,333 clock cycles by 160 columns. However, realistically, the real allowance is smaller since the game logic, player input handling, pixel output, and memory stalls must also fit in the frame. At a 4 MHz clock, the theoretical allowance would be about 1667 cycles per column. Since the DDA algorithm may require many steps to hit a wall, distant or complicated views may be much slower than nearby walls.

Adding textured walls and moving sprites will require more processing power from the Am9080. For textured walls, the CPU must determine which pixels from the texture need to be displayed and write that pixel to a frame buffer. If the textures are stored in different memory banks, the CPU may also need to perform bank switching to access them. Also, sprites will require additional calculations to determine their position and size on the screen, along with checking whether they should appear in front of or behind a wall. For our initial implementation, we plan to start with solid colored walls and few basic sprites to reduce the number of calculations required during rendering. Once we have a working raycasting game, we can measure its performance and determine whether more detailed textures and additional graphical features can be added. Since our goal is to create a simplified version of DOOM on an 8-bit processor, we do not need to implement all the graphical features found in the original version of the game.

Another factor we need to consider is how the game will be displayed on the VGA monitor. During development, we will use the SDL2 library with the obtained emulator to display the game on a computer. However, when running the game directly on the FPGA, we will need a VGA controller to generate the video signals and display the pixel data that is stored in memory. Unlike SDL2, which handles the backend and displays the images for us, the FPGA will require additional hardware to accomplish this. One possible approach is to use 2 frame buffers. While the VGA controller displays the current frame, the CPU can work on rendering the next frame in a separate memory location. Once the new frame is completed, the system switches which frame buffer is being displayed. This could help prevent incomplete frames from appearing on the screen. However, this would require additional memory and also coordination between the CPU and VGA controller. Whether we implement this method will depend on our available memory and how the frame buffer is accessed. If the CPU has not finished rendering the next frame, the VGA controller can continue displaying the previous frame until the new one is ready.

Table 3.3.4.3.2 Comparison of raycasting operations on different systems

Criteria	DirectX 12	OpenGL	Custom Raycasting
Primary role	GPU Graphics API	Graphics API	Rendering Algorithm
Requires host graphics driver/GPU model	Yes	Yes	No
Executes directly as an 8080 renderer	No	No	Can be made to target 8080 class
Compatible with bare metal applications	No	No	Yes, with custom display interface
Can use fixed-point integer calculations	No	No	Yes
Matches small grid based levels	More capability than needed	More capability than needed	Sufficient
Main challenge	Software and Hardware stack are not supported	Software and Hardware stack are not supported	CPU cycles and memory bandwidth

Commercial game engines such as Unreal Engine were also considered but were rejected early on when considering the system architecture. Current Unreal Engine development targets a 64-bit computer with substantial main memory and graphics hardware. Epic's recommended windows configuration includes 32 GB of RAM and a DirectX 12 compatible graphics card. These requirements describe the development environment rather than the exact minimum needed for every game. However, they show the difference between that software system and a processor limited to 64 KB of addressable memory. Using an engine with such requirements is essentially impossible considering our intended FPGA implementation.

Our selected rendering method is a custom raycaster written in C with integer and fixed-point arithmetic. It will use the emulator's SDL2 window only in the development phase. On the FPGA, the program will send the same logical image data to the custom video interface. In the development phase, we will begin with a simple untexted environment, followed by player movement, more complex map geometry, grayscale shading, and sprites. Our team will compare the instruction cycles for each stage and remove features that cannot be supported by the

available CPU and memory bandwidth. This gives the game a realistic path from a simple working demonstration to a more closer version to the original DOOM game.

3.3.3.4 Software Conclusions

The conclusions of the software research are consistent across the different technology categories. The C and assembly languages offer a direct programming route to develop our game for the Am9080. We will first use an 8080 emulator with SDL2 to develop and test the game on a computer before running it on our FPGA implementation. Since our project does not require an operating system, we will use bare metal programming to allow the program to run directly on the microprocessor. We also plan to use bank switching with a 16 KB memory window to explain the available memory to 4 MB while keeping the original 16-bit address bus. For the graphics, we chose raycasting with fixed-point arithmetic since the Am9080 does not support hardware multiplication, division, or floating-point operations. This approach will allow us to create a simplified DOOM inspired game without relying on modern graphics libraries or game engines. Once the software is running on the FPGA, we will test its performance and determine whether additional optimizations are needed to meet our target frame rate.

Chapter 4 | Standards and Design Constraints

Chapter 6 | Hardware Design

Chapter 7 | Software Design

Chapter 8 | Risk Assessment

Chapter 11 | Administrative Content

11.1 Budgeting & Financing

This project is sponsored by AMD and will provide us with the necessary equipment on a budget of $1500. Here is our initial Bill of Materials. As you can see, the FPGA development board serves as the core brain platform of our project (see more details in our Hardware Diagram).

Table 3. Bill of Materials

Component	Subsystem	Quantity	Unit Cost	Total
Nexys A7 AMD Artix™ 7 FPGA Trainer Board	All	1	$350.00	$350.00
HyperRAM PMOD	RAM	1	$15.00	$15.00
5V Power Supply/Adaptor	Power	1	$10.00	$10.00
Pmod Interconnect Headers	Connection	~4-5	$1.20	$6.00
Standard USB Keyboard	User Input	1	~$10.00	~$10.00
Standard USB Mouse	User Input	1	~$10.00	~$10.00

VGA Monitor	Display	1	~$60.00	~$60.00
VGA Cable	Display Communication	1	~$10.00	~$10.00
Total Cost				~$471.00

11.2 Initial Project Milestones

These tables depict the initial project milestones for Senior Design 1 & 2. These are subject to should some parts of the project get done early or later than expected.

Table 4.1 Senior Design 1 Project Milestones

Milestone	Start Date	Anticipated End Date	Duration
Project Proposal	9/07/2026	9/10/2026	1 Week
Revise Proposal	9/14/2026	9/18/2026	1 Week
30-Page Milestone	9/18/2026	10/02/2026	2 Weeks
60-Page Milestone	10/02/2026	10/16/2026	2 Weeks
90-Page Milestone	10/16/2026	10/30/2026	2 Weeks
120-Page Milestone	10/30/2026	11/13/2026	2 Weeks
Final Draft	11/13/2026	11/20/2026	1 Week

Table 4.2 Senior Design 1 Project Design Milestones

Milestone	Start Date	Anticipated End Date	Duration
Software 8-bit DOOM research	09/28/2026	10/09/2026	2 Weeks
Implementation of C	09/17/2026	09/25/2026	1 Weeks

program on an i8080 emulator			
Implementation of wall graphical rendering	10/22/2026	11/26/2026	5 Weeks
Am9080 Design research	09/28/2026	11/06/2026	5 Weeks
Am9080 partial RTL Design	11/09/2026	12/04/2026	4 Weeks
Output Display Research	09/28/2026	10/02/2026	1 Week
Basic Output Display Demo	10/05/2026	10/12/2026	1 Week
Bank Switching Research	10/05/2026	10/19/2026	2 Weeks
Memory Controller Initial Design	10/26/2026	11/09/2026	3 Weeks

Table 4.3 Senior Design 2 Project Design Milestones

Milestone	Start Date	Anticipated End Date	Duration
DOOM Software Debugging	01/11/2027	01/18/2027	1 Week
RTL Verification	01/18/2027	02/01/2027	2 Weeks
Overall System Testing	02/01/2027	02/15/2027	3 Weeks
Project Practice Demo	03/08/2027	03/19/2027	2 Weeks
Finalize Documentation	03/19/2027	04/02/2027	2 Weeks

Practice Presentation	04/09/2027	04/16/2027	1 Week
Final Presentation	TBD	TBD	TBD